

Qwen3-Coder-Next Tier 3

Part 4 Detailed Implementation Specification

Research Layer

Document role

This is an engineering specification for implementing the Research Layer from scratch.

It is intentionally narrower than the architecture document and deeper than the roadmap.

The goal is to give an engineer enough detail to build the part without inventing the missing structure.

Assumptions: architecture is fixed, filesystem tooling exists, and the planner can emit a research task.

Part 4 of the Tier 3 implementation series

1. Purpose

- The Research Layer converts an ambiguous task into verified context that the Coding Agent can trust.
- It solves the classic failure mode where an agent starts writing code before it understands the repo, the docs, the APIs, or the error surface.
- It is the read-only discovery layer between planning and execution, and later parts depend on its structured output rather than raw tool logs.

Design rule

Research may collect, rank, normalize, and summarize.

Research may not modify source code, rewrite plans, or decide deployment outcomes.

If the task needs more work than evidence gathering, it must hand the issue back to the planner or escalate a clarification request.

2. Scope

Included

- Task query normalization and research request shaping.

- Source selection across local repository files, docs, error traces, and approved external references.
- Read-only repository scanning, file sampling, snippet extraction, and document loading.
- Evidence ranking, citation attachment, and confidence scoring.
- A research packet that can be handed to the Coding Agent without further cleanup.
- Cache-aware retrieval of previously gathered evidence for repeat queries.

Excluded

- Decomposing the product task into implementation steps, which belongs to the Planning Layer.
- Generating or editing production code, which belongs to the Coding Layer.
- Persistent long-term project memory, which is introduced later.
- Knowledge graph construction and relationship traversal, which come in later parts.
- Benchmark governance and deployment gating, which belong to later quality and release layers.

3. System Overview

User task or bug report

- > LangGraph orchestrator
- > Planner emits research request
- > Research Layer selects sources and gathers evidence
- > Evidence is normalized and ranked
- > Research packet is produced with citations and confidence
- > Coding Agent consumes the packet and writes changes
- > If evidence is insufficient, planner receives a clarification or follow-up request

- The important boundary is that the Research Layer ends with a decision-quality packet, not a code artifact.
- The packet must be small enough to fit the coding context window but rich enough to avoid blind implementation.
- Read-only operations are the default; any unsafe or ambiguous source request should be blocked or downgraded.

Flow summary

Planner asks what evidence is needed.

Research gathers evidence.

Coding consumes evidence.

No part of this stage owns the final implementation.

4. Internal Architecture

ResearchRequest -> Query Interpreter -> Source Policy Engine

-> Local Repository Scanner

-> Document / Error Fetchers

-> Evidence Normalizer

-> Evidence Ranker

-> Research Synthesis Builder

-> Citation / Artifact Assembler

-> ResearchPacket output

ResearchState is updated at each stage and written to the session store / cache layer.

Core components

Query Interpreter: Rewrites a task into search intents, keywords, code symbols, error signatures, and source hints.

Source Policy Engine: Decides which sources are allowed, useful, or too risky for the current task.

Local Repository Scanner: Reads folders, file lists, README files, test failures, and targeted code snippets without mutating anything.

Document / Error Fetchers: Pulls docs, API references, issue logs, stack traces, and command output into a shared evidence stream.

Evidence Normalizer: Cleans and standardizes gathered material into a consistent internal shape.

Evidence Ranker: Scores evidence by task relevance, freshness, precision, and redundancy.

Research Synthesis Builder: Turns ranked evidence into a concise answerable packet for the next agent.

Citation / Artifact Assembler: Attaches stable references, source labels, and traceable snippets to the final packet.

Required abstractions

- **ResearchRequest:** the normalized request handed in from planning.
- **SourceHandle:** a typed pointer to a local file, repository folder, doc URL, or error artifact.
- **EvidenceItem:** a single reusable fact, snippet, excerpt, or error detail with provenance.
- **ResearchPacket:** the final packaged result that downstream agents consume.
- **SourcePolicy:** the ruleset that governs which sources are legal, preferred, or blocked.

5. Project Structure

Folder	Purpose
research/orchestrator/	Entry point that coordinates the end-to-end research flow.
research/policy/	Source allowlists, ranking rules, and safety constraints.
research/connectors/	Adapters for local files, docs, logs, and approved external fetches.
research/scanner/	Repo walking, file sampling, snippet extraction, and path filtering.
research/normalization/	Transforms raw tool output into structured evidence items.
research/ranking/	Relevance scoring, deduplication, freshness, and precision filters.
research/synthesis/	Produces the final research packet and summary narrative.
research/cache/	Cache lookup, persistence, invalidation, and reuse of evidence.
research/state/	Session-local research state objects and transition helpers.
research/schemas/	Request, evidence, packet, and policy schemas.
research/prompts/	Prompt templates for source classification, query rewriting, and synthesis.
research/observability/	Structured logging, traces, counters, and audit hooks.
research/tests/	Unit, integration, and regression tests for all of the above.
config/	Research-specific configuration files and environment mapping.

6. Data Contracts

Contract principle

The Research Layer should exchange structured objects, not raw transcripts.

Downstream agents must be able to trust provenance, source type, and confidence without re-running the research flow.

6.1 ResearchRequest

```
{
  request_id: "req-..."
  task_type: "feature" | "bug" | "investigation"
  target_repo: "..."
  query_text: "..."
  constraints: [read-only, local-only, approved-external]
  hints: [file paths, symbols, error strings, docs]
  budget: { time_ms, source_limit, snippet_limit }
}
```

6.2 EvidenceItem

```
{
  evidence_id: "evi-..."
  source_type: "repo_file" | "doc" | "log" | "api_ref"
  source_ref: "path / url / artifact id"
  excerpt: "short extracted content"
  relevance_score: 0.0-1.0
  confidence: 0.0-1.0
  freshness: "current" | "stale" | "unknown"
  provenance: { tool, timestamp, line_range }
}
```

6.3 ResearchPacket

```
{  
  request_id: "req-..."  
  summary: "concise answer to the planner question"  
  open_questions: [ ... ]  
  evidence: [EvidenceItem, ...]  
  recommended_next_action: "code" | "clarify" | "expand_research"  
  confidence: 0.0-1.0  
  citations: [stable refs]  
  artifacts: [paths to manifests / notes]  
}
```

6.4 Configuration format

```
research:  
  allowed_sources: [repo, docs, logs, approved_web]  
  default_time_budget_ms: 30000  
  max_evidence_items: 20  
  max_snippet_chars: 1200  
  cache_ttl_minutes: 60  
  source_rank_weights:  
    repo_file: 1.0  
    error_log: 0.95  
    api_ref: 0.85  
    web_doc: 0.70
```

7. State Management

- Research state starts as a request and ends as a packet, with every meaningful transition recorded.

- The orchestrator owns the live session state; the cache owns reusable evidence; the packet owns the handoff to downstream agents.
- State changes should be append-only for evidence and idempotent for cache lookups so repeated requests are safe.
- The Coding Agent should consume the research packet and not depend on hidden intermediate structures.

State transitions:

RECEIVED -> CLASSIFIED -> SOURCES_SELECTED -> EVIDENCE_GATHERED
-> EVIDENCE_NORMALIZED -> PACKET_BUILT -> HANDOFF_READY

If evidence is insufficient:

RECEIVED -> CLASSIFIED -> NEEDS_CLARIFICATION

8. Component Specifications

Component	Purpose	Inputs	Outputs	Failure conditions	Integration points
Query Interpreter	Turns task text into research intents.	Planner task, hints, budgets.	Normalized research request.	Ambiguous or underspecified tasks.	Feeds Source Policy Engine.
Source Policy Engine	Chooses permitted and useful source classes.	Normalized request, repo metadata.	Allowed source set.	Unsafe source request or contradictory constraints.	Feeds connectors and scanner.
Local Repository Scanner	Collects local repo evidence.	Allowed repo roots, path hints.	Files, snippets, metadata.	Unreadable files, missing paths, oversized scans.	Feeds Evidence Normalizer.
Document / Log Fetcher	Collects docs and error traces.	Doc refs, logs, command output.	Raw evidence artifacts.	Fetch failure, stale artifacts, malformed logs.	Feeds Evidence Normalizer.
Evidence Ranker	Ranks and deduplicates evidence.	Normalized evidence set.	Ordered evidence list.	Too many near-duplicates, low-confidence evidence.	Feeds Synthesis Builder.

Research Synthesis Builder	Creates the final packet.	Ranked evidence, request, policy.	ResearchPack et.	Overlong packet, missing citations, unresolved questions.	Handed to Coding Agent / Planner.
Cache Manager	Reuses prior evidence safely.	Request fingerprint, evidence keys.	Cache hit or miss result.	Stale cache, collision, invalid invalidation.	Used before source fetchers.
Observability Layer	Makes behavior inspectable.	State transitions, timings, decisions.	Logs, traces, counters.	Missing audit trail or unstructured output.	Cross-cuts every component.

9. Implementation Sequence

Step 1: Define schemas and state objects

Why now: This comes first because every later module depends on stable request, evidence, and packet shapes.

Expected output: Create the request, evidence, packet, policy, and state definitions.

Step 2: Build source policy and request normalization

Why now: The layer must know what to ask for before it can fetch anything.

Expected output: Convert planner output into structured source intents.

Step 3: Implement local repository scanning

Why now: Local code and docs are the highest-value sources and should be available early.

Expected output: Walk files, sample content, and extract focused snippets.

Step 4: Add document, log, and error fetchers

Why now: Research is not just repo browsing; the layer must explain failures too.

Expected output: Capture command output, stack traces, docs, and issue notes.

Step 5: Add evidence normalization and ranking

Why now: Raw fetches are too noisy to hand directly to the Coding Agent.

Expected output: Standardize and order evidence by usefulness.

Step 6: Build the synthesis packet generator

Why now: The system needs a final deliverable, not a pile of snippets.

Expected output: Produce a concise packet with citations and open questions.

Step 7: Add cache and reuse logic

Why now: Repeated tasks should be cheaper and more consistent after the first run.

Expected output: Store and retrieve evidence by fingerprint.

Step 8: Add observability and failure paths

Why now: Without traceability, debugging the research layer becomes guesswork.

Expected output: Emit structured logs, counters, and fallback decisions.

Step 9: Close the loop with integration tests

Why now: Only after the flow works end-to-end does the layer become trustworthy.

Expected output: Verify planner -> research -> coding handoff on sample tasks.

10. Validation Strategy

- Unit tests should cover normalization, policy decisions, ranking, cache key generation, and packet assembly.
- Integration tests should use a small repo fixture, a docs fixture, and at least one error-log fixture.
- Regression tests should replay known research tasks and verify that the same evidence remains reachable after refactors.
- Every test should check both content quality and structural guarantees such as provenance, source type, and packet size.

Example validation matrix:

- Given a bug report, fetch the failing file, related test, and stack trace.
- Given a feature request, fetch README, API docs, and relevant modules.
- Given an insufficient query, return clarification instead of hallucinating evidence.
- Given the same request twice, use cache and return an equivalent packet.

11. Deliverables

- Research request and packet schemas.
- Source policy configuration and ranking rules.
- Repository scanner and evidence fetchers.
- Evidence normalization and ranking pipeline.
- Research packet builder with citation support.
- Cache manager and invalidation logic.
- Structured logging and trace events.
- Unit, integration, and regression tests.
- Sample fixtures for repo, docs, and error investigations.

12. Acceptance Criteria

Criterion	Measurement	Verification method
Planner task can be normalized into a research request	100% of sample requests produce a valid request object	Schema validation test
Repository research can retrieve relevant local evidence	At least 3 relevant evidence items for fixture tasks	Integration test
Evidence items retain provenance	Every packet item includes source type and source reference	Packet inspection test
Research packets stay within size budget	Packet length below configured maximum	Length assertion
Clarification path works	Underspecified requests return a clarification state, not fabricated evidence	Negative-path test
Cache reuse is functional	Repeated request returns cache hit and equivalent packet	Two-run regression test
No write operations occur during research	Filesystem mutation count remains zero in research tests	Sandbox audit

13. Common Failure Modes

- Collecting too much raw text and starving the coding context window.
- Treating search output as evidence without checking provenance or freshness.
- Mixing planning responsibilities into the research layer because it is tempting and wrong.
- Caching stale artifacts and then pretending the result is still valid.
- Letting unsafe source access slip through because policy checks were added as decoration.
- Producing a summary that sounds neat but cannot be traced back to source snippets.

14. Future Integration

- Part 5 Memory System will persist the useful research artifacts and let future tasks reuse prior findings.
- Part 6 Testing and Review will consume research packets to validate that code matches the investigated requirement.
- Part 9 Repository Intelligence will improve path selection and source prioritization using structural repo awareness.
- Part 10 Code Knowledge Graph will let research answer relationship questions instead of only file-level questions.
- Part 11 Context Compression will depend on the evidence selection rules established here.
- Part 12 Multi-Agent Coordination will route specialized sub-research tasks through the same packet format.

15. Completion Checklist

- Schemas for request, evidence, packet, and policy are implemented and validated.
- Local repository scanning works on the reference fixture set.
- Document and error-log fetchers return normalized evidence.
- Ranking and synthesis produce a concise, citation-backed packet.
- Cache behavior is deterministic and test-covered.
- Every output contains provenance data.
- Integration tests demonstrate planner -> research -> coding handoff.
- Negative tests show that unsafe or underspecified requests do not fabricate evidence.
- No production code path performs filesystem mutation during research.

End of Part 4 specification